Práctica 1
MarioBros

Tecnología de la Programación I
Curso 2024/2025

Arquitectura general de la plantilla

- PseudoModelo-Vista-Controlador
  - La **Vista** se corresponde con el paquete *view*. Ahí se encuentran las clases encargadas de mostrar el juego, en concreto la clase**GameView*.
  - El **Controlador** se corresponde con el paquete *control*. Ahí se encuentra la clase *Controller*, encargada de gestionar el "ciclo de vida" del juego
  - El **Modelo** se corresponde con el paquete *logic*, donde se encuentra la lógica del juego, es decir, las reglas del juego y el manejo de todos los objetos del juego

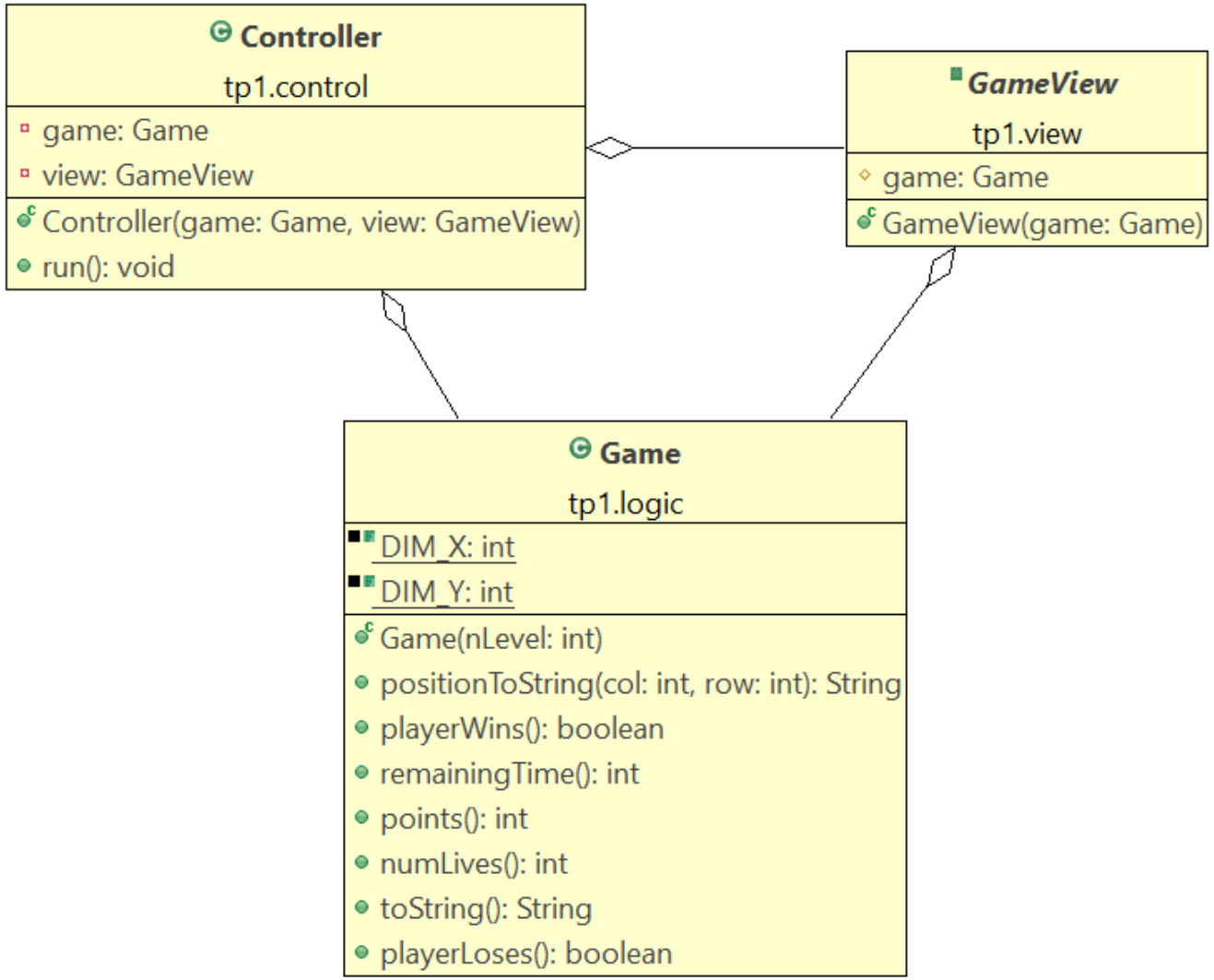
* Veremos que de hecho se trata de una interfaz

Puesta en marcha en *Main*

```
Game game = new Game(nLevel);
GameView view = args.length>1 ? new ConsoleView(game):
                                new ConsoleColorsView(game);
Controller controller = new Controller(game, view);

controller.run();
```

Puesta en marcha en *Main*



Vista: Métodos de *GameView*

```
// show methods
public void showWelcome();
public void showGame();
public void showEndMessage();
public void showError(String message);
public void showMessage(String message);


// get data from view methods
public String[] getPrompt();
```

Controlador: clase *Controller*

Implementa el método run(), invocado desde Main, que ejecuta una partida

```
public void run() {
    view.showWelcome();
    view.showGame();
    while (!game.isFinished()) {
        // Pedir una línea al usuario (getPromt)
        // Ejecutar el comando del usuario (método)
    }
    view.showEndMessage();
}
```

Modelo: clase *Game*

- La clase *Game* encapsula la lógica del juego
- Habrá una única instancia de *Game* en el programa
- Contiene una instancia de *GameObjectContainer*
- También mantiene el contador de tiempo, de puntos y de vidas …

Contenedor de objetos: *GameObjectContainer*

Clase que almacena y gestiona todos los objetos presentes en el tablero

```
private List<Land> landList;
private List<Goomba> goombaList;
private ExitDoor exit;
private Mario mario;
```

```
…
landList = new ArrayList<>();
```

- Tiene métodos para:
  - manipular esas estructuras de datos: p.ej., para añadir un nuevo Goomba
  - redirigir a los objetos peticiones provenientes de *Game*

Clase Mario

- Atributos: posición, un booleano que indica si es grande o no, la dirección de su movimiento, …
- Además, atributo *Game*(interacciones con el entorno)
- Algunos métodos:

`public void update()`: actualiza a Mario (y quizás su entorno)

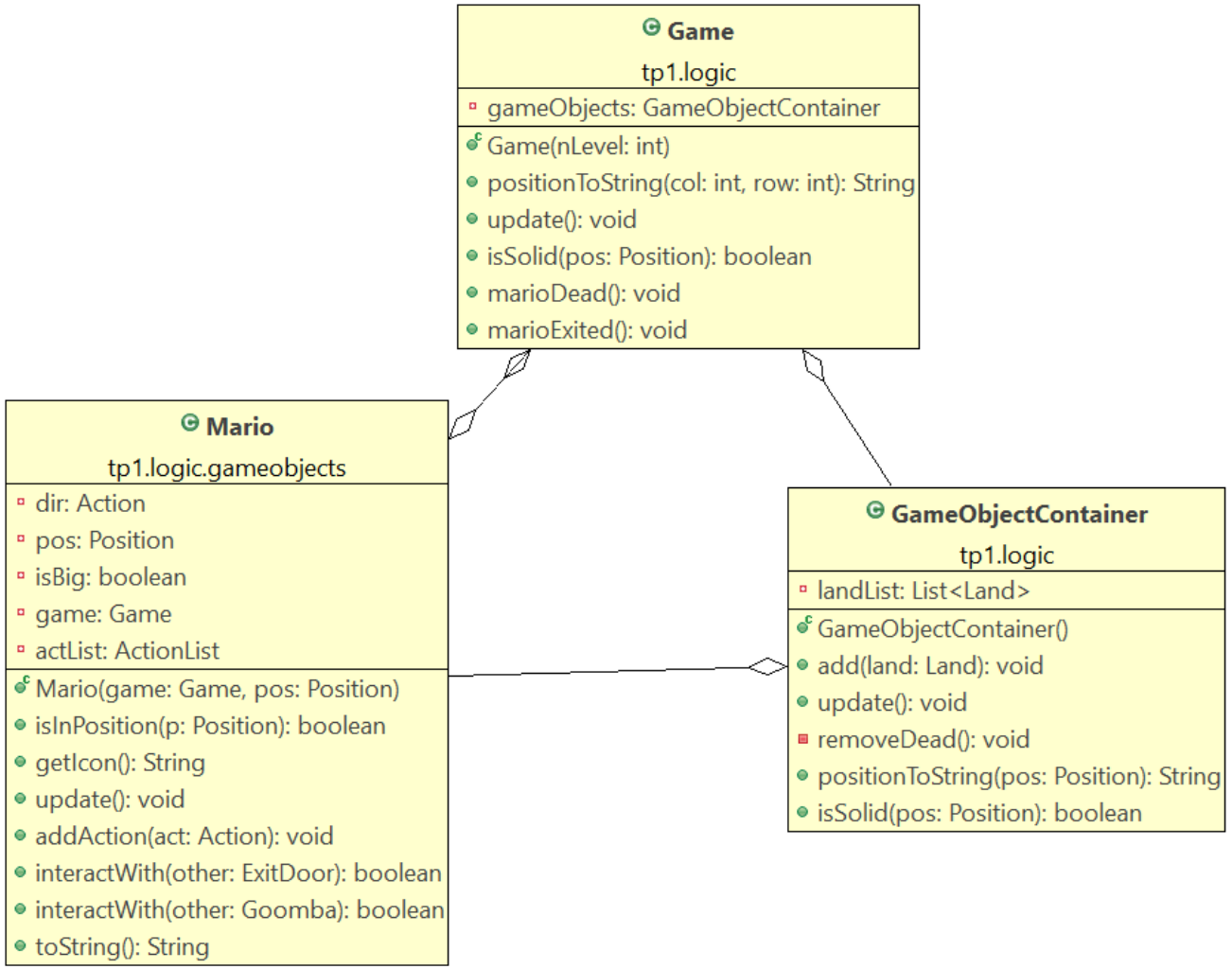
`public String getIcon()`: devuelve el icono, 🧍 , 🚶 o 🧑 , según su dirección

`public String toString()`: devuelve una representación de Mario

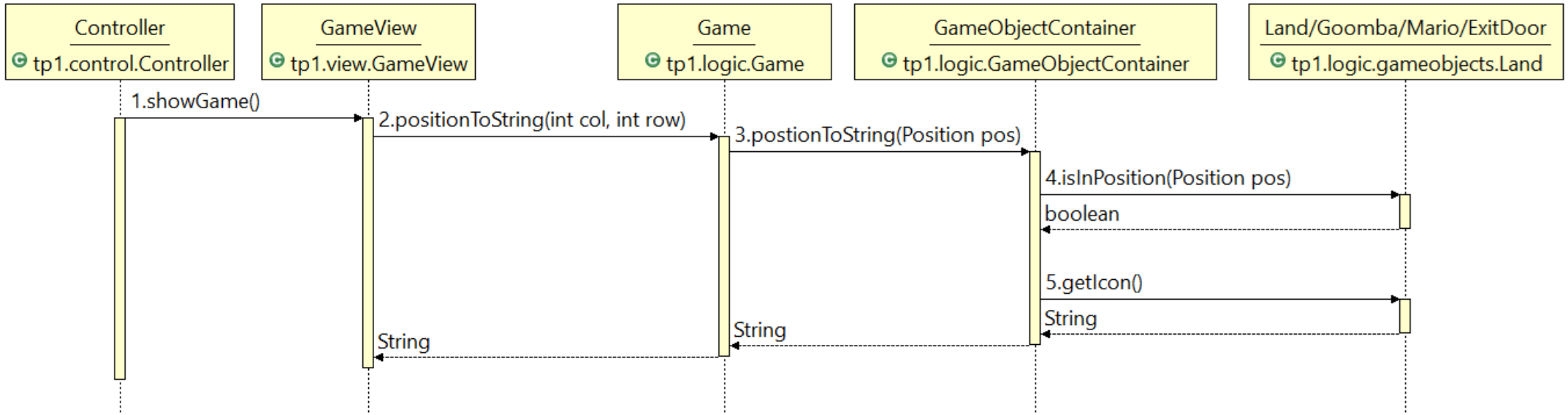
- Ej.: (1,2) Mario BIG STOP

```
Mario grande situado en la posición (1,2), parado
```

Diagrama simplificado del modelo



Secuencia de llamadas para mostrar el juego



RECORDATORIOS IMPORTANTES

- Cualquier rotura de la encapsulación será motivo de suspenso
- Ejemplos de rotura de la encapsulación:
  - Atributos públicos
  - Métodos *getters* públicos que devuelven objetos
  - Métodos públicos que devuelven listas o arrays
- *Game* no debería tener ningún bucle
  - Todos los recorridos sobre objetos del juego se realizan en *GameObjectContainer*
- No se debe añadir ningún System.out…, ni en *Controller* ni en *Game*, ni en ninguna otra clase (todos están ya en la vista).